

Plan de POC en Port

Crear repositorio y configurar pipeline
(Fases, épicas e historias de usuario)



Basado en el diagrama de secuencia UML del cliente
Orquestación: Port Workflows · IaC: Terraform vía GitHub Actions
Idioma del documento: Español

1. Objetivo y enfoque del POC

Objetivo: Demostrar un camino de extremo a extremo que un desarrollador pueda ejecutar desde Port, con aprobación de DevOps y estado de vuelta al solicitante — no paridad completa de producción el primer día.

Decisión de diseño clave para el POC: Usar el nodo nativo INPUT de Port para la puerta de aprobación Terraform plan→apply (alineado con “DevOps aprueba en Port” en el diagrama). Mantener Issues de GitHub como historia opcional posterior si el cliente exige seguimiento basado en issues.

Estrategia de cortes: Construir primero un corte vertical (addRepository) y luego reutilizar el mismo esqueleto plan→aprobar→apply para equipos y variables de entorno. Tratar addWorkflow (merge de PR) como un corte final separado.

2. Fases de implementación

Fase 0 — Fundamentos

1. Conectar Port ↔ GitHub (Ocean / aplicación de GitHub).
2. Crear un repositorio de control de plataforma con módulos Terraform y GitHub Actions workflow_dispatch (plan / apply).
3. Definir blueprints mínimos: service (o reutilizar el integrado), githubRepository (de la integración), provisioningRequest opcional para seguimiento de ejecuciones.
4. Guardar credenciales de Port como secretos de GitHub; usar port-labs/port-github-action para reportar el estado de la ejecución.

Fase 1 — Corte vertical: Crear repositorio

1. Formulario de self-service: nombre del repo, visibilidad, equipo, plantilla.
2. INTEGRATION_ACTION → plan de GitHub Actions (Terraform crea el repo).
3. Nodo INPUT → DevOps aprueba / rechaza.
4. Si se aprueba → workflow de apply; si tiene éxito → upsert de entidad en Port y notificación al usuario.
5. Prueba manual con una organización / sandbox desechable.

Fase 2 — Reutilizar el patrón: Añadir equipo

1. Segunda acción de self-service (o paso encadenado tras existir el repo).
2. Mismo ciclo plan→aprobar→apply sobre Terraform de membresía de equipo.
3. Relacionar repo ↔ equipo en Port (\$team / relations).

Fase 3 — Pipeline y variables de entorno

1. Formulario para tipo de pipeline y claves/valores de variables de entorno (secretos vía GitHub Environments / APIs de secretos de Actions, detrás de Terraform o un GHA dedicado).
2. Mismo ciclo de aprobación; upsert de metadatos de pipeline/env en la entidad service.

Fase 4 — Añadir archivos de workflow (puerta por PR)

1. Port despacha una acción que abre un PR con .github/workflows/* (o genera desde una plantilla).
2. DevOps hace merge en GitHub (puerta humana fuera de Port).
3. Trigger de evento / polling / callback de estado actualiza Port cuando el PR se fusiona.
4. Notificación final de estado al solicitante.

Fase 5 — Unificar el recorrido (opcional para el POC)

1. Un workflow multi-paso con etapas secuenciales, o cuatro workflows más un orquestador ligero que los encadene vía eventos de estado de entidad.

2. Dashboard: solicitudes de aprovisionamiento, aprobaciones pendientes, catálogo de repositorios.

3. Orden de construcción sugerido

1. A1-A3 + B1-B4 — demo mínima alineada con la Fase 1 del UML.
2. C1-C2 — demuestra que el patrón es reutilizable.
3. D1-D3 — demuestra secretos y configuración de pipeline.
4. E1-E3 — demuestra el mecanismo de aprobación distinto (PR).
5. F1-F2 — pulido para la demo a stakeholders.

4. Épicas e historias de usuario

Épica A — Fundamentos de plataforma

A1 - Como ingeniero de plataforma, conecto Port a nuestra organización de GitHub para que los workflows puedan despachar Actions.

Criterios de aceptación: Integración de GitHub saludable; se puede despachar un workflow_dispatch vacío desde Port y ver éxito.

A2 - Como ingeniero de plataforma, tengo un repo de control con Terraform + Actions plan/apply que reportan estado a Port.

Criterios de aceptación: El plan sube un artefacto; apply lo consume; ambos hacen PATCH de logs de ejecución en Port vía port-github-action.

A3 - Como ingeniero de plataforma, defino blueprints para service / estado de aprovisionamiento.

Criterios de aceptación: La entidad puede rastrear status (requested → planned → approved → applied / failed) y enlazar a la URL del repo de GitHub.

Épica B — Crear repositorio (addRepository)

B1 - Como desarrollador, envío “Crear repositorio” desde Port con nombre, visibilidad y equipo propietario.

Criterios de aceptación: El formulario valida entradas; la ejecución aparece en Port con la identidad del solicitante.

B2 - Como sistema, ejecuto Terraform plan para el nuevo repo y pausamos por aprobación.

Criterios de aceptación: El plan tiene éxito; DevOps ve una aprobación INPUT pendiente con resumen/enlace del plan; aún no hay apply.

B3 - Como DevOps, apruebo o rechazo el cambio en Port.

Criterios de aceptación: Aprobar → se ejecuta apply; rechazar → la ejecución termina, se notifica al usuario, no se crea el repo.

B4 - Como desarrollador, recibo el estado cuando termina el aprovisionamiento.

Criterios de aceptación: El repo existe en GitHub; la entidad en Port se actualiza; el status SUCCESS/FAILURE coincide con la realidad.

Guion de prueba del POC (B)

- Camino feliz
- Camino de rechazo
- Fallo del plan (nombre inválido)

Épica C — Añadir equipo (addTeam)

C1 - Como desarrollador, solicito acceso de equipo a un repo existente desde Port.

Criterios de aceptación: El formulario selecciona entidad de repo + equipo; solo se muestran equipos permitidos.

C2 - El mismo ciclo plan→aprobar→apply aplica permisos de equipo vía Terraform.

Criterios de aceptación: Tras aprobar, el equipo de GitHub tiene el permiso esperado; relations/\$team en Port actualizados.

Guion de prueba del POC (C)

- Añadir equipo
- Rechazar
- Intento sobre un repo que el usuario no posee (chequeo de permisos)

Épica D — Pipeline y variables de entorno (addPipeline / addEnvVars)

D1 - Como desarrollador, elijo una plantilla de pipeline y variables de entorno para un repo.

Criterios de aceptación: El formulario captura no-secretos + referencias a secretos; los secretos nunca se registran en la salida de la ejecución de Port.

D2 - Plan/apply configura GitHub Environment / variables vía Terraform o GHA.

Criterios de aceptación: Las variables están presentes en GitHub; Port muestra “pipeline configurado”.

D3 - Se requiere aprobación de DevOps antes de escribir secretos/variables.

Criterios de aceptación: Sin mutación hasta aprobar.

Guion de prueba del POC (D)

- Solo entorno de staging
- Verificar redacción de secretos en los logs

Épica E — Añadir archivos de workflow (addWorkflow)

E1 - Como desarrollador, solicito el andamiaje de workflow de CI para mi repo.

Criterios de aceptación: Port crea una rama + PR con YAML de workflow desde una plantilla.

E2 - Como DevOps, reviso y hago merge del PR en GitHub.

Criterios de aceptación: El merge es la puerta de aprobación (como en el UML); sin push silencioso a la rama por defecto.

E3 - Como desarrollador, veo el estado final en Port tras el merge.

Criterios de aceptación: El evento de PR fusionado (o polling) actualiza la entidad; se notifica al usuario.

Guion de prueba del POC (E)

- Abrir PR → merge → Port SUCCESS
- Abrir PR → cerrar sin merge → FAILED/CANCELLED

Épica F — UX del recorrido y observabilidad (ligera para el POC)

F1 - Como desarrollador, puedo ver dónde está atascada mi solicitud (plan / aprobación / apply / PR).

Criterios de aceptación: Una línea de tiempo de ejecución o la entidad provisioningRequest muestra la etapa.

F2 - Como DevOps, tengo una cola de aprobaciones pendientes.

Criterios de aceptación: Una página/widget de Port lista las aprobaciones INPUT abiertas.

5. Fuera de alcance del POC

- Aprobación completa basada en GitHub Issues espejando el UML (usar Port INPUT primero).
- Backends Terraform multi-organización / multi-nube.
- Encadenamiento automático de las cuatro fases en un solo clic sin confirmación intermedia.
- Endurecimiento RBAC de producción más allá de “Member puede solicitar / equipo DevOps puede aprobar”.

6. Mapeo UML → constructos de Port

- Usuario addRepositorie(form) → Workflow SELF_SERVE_TRIGGER
- Port → GithubAPI createRepo → Indirecto vía Terraform en GHA (preferir TF por paridad plan/apply)
- Terraform execute(plan) / execute(apply) → Dos Actions workflow_dispatch + INTEGRATION_ACTION
- Issues wait(approval) + DevOps sendApprove → Nodo INPUT de Port (POC); opcional después: GitHub Issue + callback
- waitUntilEnd / sendStatus → reportWorkflowStatus + port-github-action PATCH_RUN / upsert de entidad
- addWorkflow + createPullRequest + Merge → GHA abre PR; merge humano; actualización de evento/estado en Port

7. Guion de demostración

1. El desarrollador abre Crear repositorio en Port y envía el formulario.
2. Mostrar la ejecución de plan en GitHub Actions y la aprobación pendiente en Port.
3. DevOps aprueba → apply crea el repo → aparece la entidad en el catálogo.
4. El desarrollador ejecuta Añadir equipo y luego Configurar pipeline.
5. El desarrollador ejecuta Añadir workflow → se abre el PR → DevOps hace merge → Port muestra completado.

8. Riesgos a señalar temprano

- Estado asíncrono de larga duración: los artefactos del plan deben indexarse por el run ID de Port para que apply use el plan correcto.
- Dos estilos de aprobación: basado en Issues (UML) vs Port INPUT vs merge de PR — alinear con el cliente antes de construir automatización de Issues.
- Workflows están en open beta: preferir Workflows para lógica multi-paso nueva; Actions & Automations legacy siguen soportadas si ya se usan.
- Secretos: nunca poner valores secretos en salidas de ejecución de workflow ni en propiedades de entidades.